

HDOC Day-20

Application of Iterative and Decision-Making Constructs to Shapes and Physics using Modular Programming in C

Aim

To develop, compile and test a menu-driven modular C project that calculates geometrical measurements and physical properties of selected solids using functions, loops, selection statements and separate source files.

Theory / How the project is done

The program is divided into modules. **main.c** displays the menu, accepts common inputs and calls the required functions. **geometry.c** contains formulas for surface area, volume and base area. **physics.c** performs mass, weight, potential energy, pressure, buoyant-force and buoyancy calculations. **math_utils.c** implements square root by the Newton-Raphson iterative method. **solid.h** contains constants, symbolic choices and function declarations. Input is checked using loops so dimensions and densities are positive and elevation is non-negative.

Important Formulae

Quantity	Formula
Sphere area	$4 \times \text{PI} \times r^2$
Sphere volume	$(4/3) \times \text{PI} \times r^3$
Cylinder area	$2 \times \text{PI} \times r \times (r+h)$
Cylinder volume	$\text{PI} \times r^2 \times h$
Cone area	$\text{PI} \times r \times (r + \sqrt{r^2+h^2})$
Cone volume	$\text{PI} \times r^2 \times h / 3$
Cuboid area	$2(lb + bh + hl)$
Cuboid volume	$l \times b \times h$
Mass	density x volume
Weight	mass x g
Potential energy	mass x g x elevation
Pressure on base	density x g x height
Buoyant force	fluid density x g x submerged volume

Algorithm

1. Start and display the main menu repeatedly until Exit is selected.
2. Read the menu choice and reject choices outside 1 to 10.
3. For Surface Area, Volume or Base Area, display the solid submenu: Sphere, Cylinder, Cone and Cuboid.
4. Read required dimensions and repeat input while any required dimension is zero or negative.
5. Call the corresponding geometry function and display the returned value. Sphere is not permitted for Base Area.
6. For Mass, read positive density; select a solid, obtain its dimensions, calculate its volume and return density x volume.
7. For Weight, read positive mass and calculate mass x GRAVITY.
8. For Potential Energy, read non-negative elevation and positive mass; calculate mass x GRAVITY x elevation.

9. For Pressure, read positive density and height; calculate density x GRAVITY x height.
10. For Buoyant Force, read positive fluid density and submerged volume; calculate fluid_density x GRAVITY x volume.
11. For Buoyancy, compare solid density with fluid density. Lower density floats, equal density remains suspended, and higher density sinks.
12. Use Newton-Raphson iteration in math_utils.c to calculate square root for the cone slant height.
13. Repeat the menu; stop when option 10 is selected.

Project Organization

Module	Responsibility
main.c	Main menu, common input, function calls and result display
geometry.c	Surface area, volume and base-area functions
physics.c	Mass, weight, potential energy, pressure and buoyancy functions
math_utils.c	Newton-Raphson square root and absolute-value helper
solid.h	Constants, symbolic choices and external function declarations

Program Files

solid.h

```
#ifndef SOLID_H
#define SOLID_H
#define PI 3.1416
#define GRAVITY 9.8
#define TOLERANCE 0.000001
#define SPHERE 1
#define CYLINDER 2
#define CONE 3
#define CUBOID 4
#define FLOATS -1
#define SUSPENDED 0
#define SINKS 1

double sphere_area(double radius);
double sphere_volume(double radius);
double cylinder_area(double radius,double height);
double cylinder_volume(double radius,double height);
double cone_area(double radius,double height);
double cone_volume(double radius,double height);
double cuboid_area(double length,double breadth,double height);
double cuboid_volume(double length,double breadth,double height);
double circular_base_area(double radius);
double cuboid_base_area(double length,double breadth);
double calculate_mass(double density);
double calculate_weight(void);
double potential_energy(double elevation);
double calculate_pressure(double density);
double buoyant_force(double fluid_density);
int determine_buoyancy(double solid_density,double fluid_density);
double newton_sqrt(double x);
double absolute(double x);
#endif
```

main.c

```
#include <stdio.h>
#include "solid.h"

int main(void) {
    int choice, solid, state;
    double r,h,l,b,density,fluid_density,elevation,result;
    do {
        printf("\n1 Surface Area  2 Volume  3 Base Area  4 Mass  5 Weight\n6 Potential Energy  7 Pressure  8 Buoyant Force  9 Buoyancy  10 Exit\n");
        if (scanf("%d", &choice) != 1) return 0;
        if (choice == 10) break;
        if (choice < 1 || choice > 10) {
            printf("Invalid menu choice.\n");
            continue;
        }
        if (choice >= 1 && choice <= 3) {
            do {
                printf("Solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): ");
                scanf("%d", &solid);
            } while (solid < SPHERE || solid > CUBOID);
            if (choice == 3 && solid == SPHERE) {
                printf("Sphere has no base area.\n");
                continue;
            }
            if (solid == SPHERE) {
                do {
                    printf("Radius (m): ");
                    scanf("%lf",&r);
                } while(r<=0);
                result = choice==1?sphere_area(r):sphere_volume(r);
            }
            else if (solid == CYLINDER || solid == CONE) {
                do {
                    printf("Radius height (m): ");
                    scanf("%lf%lf",&r,&h);
                } while(r<=0||h<=0);
                if(choice==3) result=circular_base_area(r);
                else if(solid==CYLINDER) result=choice==1?cylinder_area(r,h):cylinder_volume(r,h);
                else result=choice==1?cone_area(r,h):cone_volume(r,h);
            }
            else {
                do {
                    printf("Length breadth height (m): ");
                    scanf("%lf%lf%lf",&l,&b,&h);
                } while(l<=0||b<=0||h<=0);
                result=choice==1?cuboid_area(l,b,h):(choice==2?cuboid_volume(l,b,h):cuboid_base_area(l,b));
            }
            printf("Result = %.4f\n", result);
        }
    }
    else if (choice == 4) {
        do {
            printf("Density (kg/m^3): ");
            scanf("%lf",&density);
        } while(density<=0);
    }
```

```

printf("Mass = %.4f kg\n",calculate_mass(density));
}
    else if (choice == 5) printf("Weight = %.4f N\n",calculate_weight());
    else if (choice == 6) {
do {
printf("Elevation (m): ");
scanf("%lf",&elevation);
} while(elevation<0);
printf("Potential energy = %.4f J\n",potential_energy(elevation));
}
    else if (choice == 7) {
do {
printf("Density (kg/m^3): ");
scanf("%lf",&density);
} while(density<=0);
printf("Pressure = %.4f Pa\n",calculate_pressure(density));
}
    else if (choice == 8) {
do {
printf("Fluid density (kg/m^3): ");
scanf("%lf",&fluid_density);
} while(fluid_density<=0);
printf("Buoyant force = %.4f N\n",buoyant_force(fluid_density));
}
    else {
do {
printf("Solid density and fluid density (kg/m^3): ");
scanf("%lf%lf",&density,&fluid_density);
} while(density<=0||fluid_density<=0);
state=determine_buoyancy(density,fluid_density);
printf(state==FLOATS?"Solid floats.\n":state==SINKS?"Solid sinks.\n":"Solid remains suspended.\n");
}
}
while (1);
printf("Program ended.\n");
return 0;
}

```

geometry.c

```

#include "solid.h"

double sphere_area(double radius) {
return 4.0 * PI * radius * radius;
}
double sphere_volume(double radius) {
return (4.0 / 3.0) * PI * radius * radius * radius;
}
double cylinder_area(double radius, double height) {
return 2.0 * PI * radius * (radius + height);
}
double cylinder_volume(double radius, double height) {
return PI * radius * radius * height;
}
double cone_area(double radius, double height) {
double l = newton_sqrt(radius*radius + height*height);
return PI * radius * (radius + l);
}
double cone_volume(double radius, double height) {
return (PI * radius * radius * height) / 3.0;
}
double cuboid_area(double length, double breadth, double height) {
return 2.0 * (length*breadth + breadth*height + height*length);
}
double cuboid_volume(double length, double breadth, double height) {
return length * breadth * height;
}
double circular_base_area(double radius) {
return PI * radius * radius;
}
double cuboid_base_area(double length, double breadth) {
return length * breadth;
}

```

physics.c

```

#include <stdio.h>
#include "solid.h"

double calculate_mass(double density) {
int solid;
double r, h, l, b, volume = 0.0;
do {
printf("Solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): ");
scanf("%d", &solid);
} while (solid < SPHERE || solid > CUBOID);
if (solid == SPHERE) {
do {
printf("Radius (m): ");
scanf("%lf", &r);
} while (r <= 0);
volume = sphere_volume(r);
}

```

```

        else if (solid == CYLINDER || solid == CONE) {
do {
printf("Radius (m): ");
scanf("%lf", &r);
} while (r <= 0);
do {
printf("Height (m): ");
scanf("%lf", &h);
} while (h <= 0);
volume = solid == CYLINDER ? cylinder_volume(r,h) : cone_volume(r,h);
}
        else {
do {
printf("Length breadth height (m): ");
scanf("%lf%lf%lf", &l,&b,&h);
} while (l<=0 || b<=0 || h<=0);
volume = cuboid_volume(l,b,h);
}
        return density * volume;
}

double calculate_weight(void) {
    double mass;
do {
printf("Mass (kg): ");
scanf("%lf", &mass);
} while (mass <= 0);
return mass * GRAVITY;
}

double potential_energy(double elevation) {
    double mass;
do {
printf("Mass (kg): ");
scanf("%lf", &mass);
} while (mass <= 0);
return mass * GRAVITY * elevation;
}

double calculate_pressure(double density) {
    double h;
do {
printf("Solid height (m): ");
scanf("%lf", &h);
} while (h <= 0);
return density * GRAVITY * h;
}

double buoyant_force(double fluid_density) {
    double volume;
do {
printf("Submerged volume (m^3): ");
scanf("%lf", &volume);
} while (volume <= 0);
return fluid_density * GRAVITY * volume;
}

int determine_buoyancy(double solid_density, double fluid_density) {
    if (absolute(solid_density - fluid_density) < TOLERANCE) return SUSPENDED;
    return solid_density < fluid_density ? FLOATS : SINKS;
}

```

math_utils.c

```

#include "solid.h"

double newton_sqrt(double x) {
    double guess, next;
    if (x <= 0.0) return 0.0;
    guess = x > 1.0 ? x : 1.0;
    do {
        next = 0.5 * (guess + x / guess);
        if (absolute(next - guess) < TOLERANCE) break;
        guess = next;
    }
    while (1);
    return next;
}

double absolute(double x) {
    return x < 0.0 ? -x : x;
}

```

Test Case Table

No.	Operation / Input	Expected Result
1	Sphere area, r=2 m	50.2656 m ²
2	Cylinder volume, r=2 m, h=3 m	37.6992 m ³
3	Cone area, r=3 m, h=4 m	75.3984 m ²
4	Cuboid base, l=5 m, b=4 m	20.0000 m ²
5	Sphere selected for Base Area	Display: Sphere has no base area
6	Dimension = 0 or negative	Reject and request input again
7	Buoyancy: solid=800, fluid=1000 kg/m ³	Solid floats
8	Buoyancy: solid=1000, fluid=1000 kg/m ³	Solid remains suspended
9	Buoyancy: solid=1200, fluid=1000 kg/m ³	Solid sinks
10	Invalid menu choice, e.g. 12	Display invalid choice and menu again

Compilation and Execution

Compile all modules together using: **gcc -std=c11 -Wall -Wextra main.c geometry.c physics.c math_utils.c -o hdoc_day20**. The supplied project was compiled successfully. A sphere surface-area test with radius 2 m produced 50.2656, matching $4 \times 3.1416 \times 2 \times 2$.

Sample Output

```
1 Surface Area  2 Volume  3 Base Area  4 Mass  5 Weight
6 Potential Energy  7 Pressure  8 Buoyant Force  9 Buoyancy  10 Exit
Choice: Solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): Radius (m): Result = 50.2656
```

```
1 Surface Area  2 Volume  3 Base Area  4 Mass  5 Weight
6 Potential Energy  7 Pressure  8 Buoyant Force  9 Buoyancy  10 Exit
Choice: Program ended.
```

Result

The modular C project successfully demonstrates iteration, switch/decision logic, validation, user-defined functions and separate compilation units while calculating the required geometry and physics quantities.